

A CONCRETE EXTENSION TO FORGE

Algebraic Witness Atlases for Forge

Checked nonlinear quantifiers, piecewise strategies,
and exact exceptional fibres

Prepared for Vladimir Reshetnikov

15 September 2026

The extension in one sentence. Replace a fixed polynomial-witness dictionary with a finite, exactly checked decomposition of the parameter line and its algebraic fibres, from which Forge can derive quantified truth, exact feasible parameter regions, and root-index witnesses or counterstrategies.

Delivered. A working two-variable Python producer and replay checker; 24 primary examples; 76 affine-transformed cases; independent oracle and mutation tests; complete JSON certificates; mathematical soundness and completeness arguments; and a specific Lean implementation plan.

Boundary. This is a new research worker for the reviewed Forge design, not an installed Lean tactic. The Python checker and geometric soundness theorem have not been formalized or checked by Lean. Classical cylindrical algebraic decomposition is the mathematical foundation, not a claimed new invention.

Reviewed Forge snapshot: `58ea206bd7ad501b930add568151217bcc7f82a2`.

Companion archive contains all source, certificate, test, and build files.

Abstract

Forge already contains a broad collection of certificate languages and proof-planning ideas. Adding another scheduler, a generic sum-of-squares recommendation, or ordinary affine witness synthesis would mostly duplicate that work. This article develops a narrower but substantial missing lane: exact bivariate real quantification with algebraic, potentially discontinuous witnesses.

For a Boolean combination $F(x, y)$ of rational polynomial sign tests, the worker handles one quantifier for y and optionally one for x . It constructs a finite sign atlas, including every exceptional parameter at which degrees drop or roots meet. Projection is justified by explicit Bézout identities rather than a trusted resultant routine. Algebraic exceptional fibres are checked with localized rational-function arithmetic at an isolated root; no irreducibility of the global projection polynomial is assumed. Complete root covers and sign tables determine existential and universal truth and provide root-index strategies.

The accompanying implementation passes 24 primary examples, 76 invertible affine reparameterizations, 540 corruption controls, and 42 unit/regression tests. Independent SymPy checks agree on 250 rational-polynomial root problems, 194 defined algebraic expressions with six zero-denominator rejections, 336 complete rational fibres, and all 25 base cells of five exact parameter-region examples. These are separate evidence units, not a pooled success rate. The checker replays saved certificates with Python site packages disabled. The remaining Lean work is identified explicitly: exact arithmetic and Sturm correctness, root continuation, the source-expression bridge, reflected checking, and strategy elaboration.

Contents

1	The capability delta, not another architectural rewrite	1
1.1	What the reviewed repository already settles	1
1.2	A witness language that is genuinely larger	2
1.3	The precise supported fragment	2
1.4	Relationship to existing mathematics and software	2
1.5	Deliverable status at a glance	3
2	Projection by checkable identities	3
2.1	What can change as the parameter moves?	3
2.2	Source-preserving factorization receipts	3
2.3	How the producer finds the receipts	4
2.4	The base decomposition	5
2.5	Why an open base cell is safe to sample	5
2.6	Exceptional points are fresh problems	6
2.7	The finite result and completeness statement	6
3	Exact algebraic points without a false field assumption	7
3.1	The representation that the checker actually needs	7
3.2	Localized rational-function coefficients	7
3.3	Exact zero first, interval sign second	8
3.4	Sturm chains and endpoint discipline	9
3.5	Complete coverage, not a plausible list of roots	9
3.6	Lifting over an algebraic base point	10
3.7	Producer/checker independence, stated precisely	10
3.8	Cost centers and useful limits	10
4	From sign tables to witnesses, counterstrategies, and formulas	11
4.1	Quantifiers are a finite game on the atlas	11
4.2	Why the sample is not the witness function	11
4.3	Worked example: a strict band of admissible squares	12
4.4	Worked example: an exact disconnected parameter region	12
4.5	Worked example: a degree drop that is not a failure	13
4.6	Worked example: cubic and quintic inverse relations	13
4.7	Worked example: discontinuity is allowed when the specification allows it	14

4.8	Worked example: quantifier order and dual evidence	14
4.9	Free-variable output and reuse by Forge	14
5	A concrete integration plan for Lean, Forge, Leant, and Djex	15
5.1	Separate the executable data model from the semantic model	15
5.2	The Lean theorem that must actually be earned	15
5.3	Reification must preserve the actual quantified variables	16
5.4	Do not turn real existence into an unpromised executable program	16
5.5	Worker scheduling and useful partial outputs	17
5.6	A specific Leant/Djex cooperation loop	17
5.7	Libraries and theorem dependencies worth reusing	17
5.8	Two concrete next algorithms, beyond the baseline prototype	18
5.9	Implementation milestones and anti-substitution acceptance tests	19
6	Executed experiments and what they establish	19
6.1	Environment, corpus, and evidence units	19
6.2	Primary acceptance and independent parameter-region checks	20
6.3	Differential arithmetic and fibre oracles	21
6.4	Metamorphic cases and replay without search libraries	21
6.5	Corruption controls	21
6.6	Observed timings, with the right denominator	22
6.7	A preserved failure and a code-review repair	22
6.8	What remains unestablished	22
A	Reproduction and certificate specification	23
A.1	Commands	23
A.2	Problem format	23
A.3	Certificate fields and their obligations	24
A.4	Checker order	24
B	Primary corpus inventory	25
C	Review record and artifact map	26
C.1	Repository snapshots	26
C.2	Files in the companion archive	26
D	Closing recommendation	27

1 The capability delta, not another architectural rewrite

1.1 What the reviewed repository already settles

The reference point is the merged Forge repository at commit `58ea206bd7ad`, not the original one-page idea. Its status ledger describes three rounds and twenty-seven contributing proposals. Its nonlinear chapter already covers exact quadratic decomposition, polynomial cone certificates, Bernstein subdivision, univariate square-factor/Sturm certificates, and cooperation with other theories. Affine, integer-lattice, residue, and polynomial witnesses are also present. Later rounds add real linear projection, noncommutative reasoning, analytic certificates, and quantitative verification. These are prerequisites, not contributions of the present article [1, 2].

The review for this extension concentrated on the complete merged nonlinear chapter, the current status and neighboring-project ledgers, the public certificate contract, and the finite-cover interface. It did not re-audit every frozen proposal or rerun Forge’s existing suites. Thus the claim is deliberately specific: this delivery implements a coherent nonlinear algebraic-quantifier lane not implemented in the reviewed merged capability inventory. It does not claim that the word “quantification” or a general aspiration to stronger arithmetic never appeared in an archived proposal.

Forge’s existing univariate worker answers a question such as whether a specified $p(t)$ is nonnegative on a specified interval. Its polynomial-witness workers search a chosen expression language. The new worker answers a different question:

$$\forall x \in \mathbb{R} \exists y \in \mathbb{R} F(x, y), \quad (1)$$

where F can combine equations, disequalities, strict inequalities, disjunctions, and implications. It can also determine exactly which x admit such a y , or refute a proposed quantifier order by producing a counterstrategy.

This distinction changes the search object. Instead of guessing one algebraic expression and trying to verify it globally, we discover the finitely many ways the admissible fibres can change, then construct a witness separately on each resulting parameter cell.

Existing ingredient	What it provides	Additional object here
Univariate root/sign checking	Signs for a fixed polynomial	Parametric root sections and exceptional fibres
Affine/polyhedral projection	Linear guards and witnesses	Nonlinear algebraic boundaries and disconnected fibres
Polynomial witness search	A successful dictionary term	Root-index and piecewise semi-algebraic selectors
Finite algebra covers	Induction over a discrete algebra	A cover of all real parameters and all fibre sectors
Certificate ownership	Association with an exact source problem	An atlas receipt occupying that existing interface

The implementation is intentionally not a replacement for cheap local saturation. A factorization, simple square, or linear elimination should still be discharged without constructing a plane decomposition. The proposed controller invokes this worker when an essential nonlinear real quantifier, a changing root structure, or an exact parameter-region request survives those cheaper routes.

1.2 A witness language that is genuinely larger

Consider

$$\forall x \in \mathbb{R} \exists y \in \mathbb{R} \quad y^2 = x^2 + 1 \wedge y \geq 0.$$

The natural answer is the positive root of $T^2 - x^2 - 1$. There is no polynomial $p \in \mathbb{R}[X]$ satisfying $p(x)^2 = x^2 + 1$ for all real x : a nonconstant solution must have degree one, say $ax + b$, and coefficient comparison forces $a^2 = 1$, $2ab = 0$, and $b^2 = 1$, which is impossible. Raising a polynomial-template degree cap cannot repair that mismatch.

The atlas does not need a radical simplifier. It selects the appropriate real root by its ordered position in the fibre’s root list. The same representation applies to $y^5 + y = x$, where a uniform radical syntax is an unnecessary restriction. It also applies when a sign condition forces the positive branch for $x \geq 0$ and the negative branch for $x < 0$. A single continuous witness is then not always the right target, whereas a finite branch-dependent selector is.

A second kind of gain is negative information. The sentence $\forall x \exists y \ xy = 1$ fails only at the isolated parameter $x = 0$. A sample away from zero is excellent heuristic evidence and entirely inadequate as a proof. A complete atlas must include that parameter as its own cell. Similarly, some feasible parameter regions consist only of isolated irrational points; no collection of rational test points can discover their membership by numerical sampling alone.

1.3 The precise supported fragment

An input consists of polynomials $p_0, \dots, p_{m-1} \in \mathbb{Q}[X, Y]$, a Boolean formula built from

$$p_i \bowtie 0, \quad \bowtie \in \{=, \neq, <, \leq, >, \geq\},$$

and quantifiers

$$Q_x x \ Q_y y \ F(x, y), \quad Q_y \in \{\exists, \forall\}, \quad Q_x \in \{\exists, \forall, \text{free}\}.$$

“Free” means that the output is the exact subset of the x -line on which the inner quantified formula holds. This is a two-stage, two-variable real-algebraic fragment. It is not the entire first-order language with two reusable variable names, and it does not support arbitrary additional symbolic real parameters. Quantified subformulas inside F , transcendental functions, integer quantification, and dependent source types are outside the delivered parser.

A bounded domain is encoded without a second mechanism. For example, $x \in [a, b]$ becomes an antecedent for universal x , or a conjunction for existential x . A bound on existential y belongs in the conjunction; a restriction on universal y belongs in an implication. That polarity distinction is part of the meaning of the input, not a solver optimization.

The implementation distinguishes a valid certificate for a true sentence, a valid certificate for a false sentence, and failure to obtain or replay a certificate. An accepted false verdict is a mathematical answer to the encoded closed sentence. A timeout, limit, or malformed certificate is not a false verdict. Without a proved Lean reification bridge, neither kind of accepted Python verdict is a Lean theorem.

1.4 Relationship to existing mathematics and software

The geometric foundation is cylindrical algebraic decomposition (CAD): partition real space into connected cells on which every relevant polynomial has constant sign, then exploit the cylindrical arrangement to propagate quantifiers. This is classical mathematics, introduced by Collins and extensively developed since; modern work also studies coverings, local cells, and proof-oriented formulations [7, 8, 9, 10].

The contribution here is an implementable Forge-specific specialization: a certificate with explicit guard identities, exact exceptional-fibre arithmetic, complete coverage checks, executable sign tables, and root-index strategy extraction. Limiting the first implementation to a parameter line makes the root-continuation argument and the exceptional-case boundary relatively transparent. It avoids pretending that the difficult nullification issues of higher-dimensional projection disappear by renaming them.

The source-level ownership and acceptance principles are inherited from Forge, Leant, and Djex, not rediscovered. Leant’s behavioral synthesis currently checks an exact rendered candidate against an exact assertion, using bounded proof attempts; the proposed atlas supplies a new proof-producing route for supported universally quantified real-algebraic assertions. Djex remains useful for constructing the surrounding higher-order term and choosing typed provider applications. Neither project needs to adopt the atlas’s polynomial representation as a substitute for its own typed source graph [3, 5, 6].

1.5 Deliverable status at a glance

Status. Executed: Python producer, exact arithmetic, replay checker, examples, differential tests, mutation tests, affine metamorphic corpus, and root-index transport probes.

Status. Specified but not implemented in Lean: polynomial reification, the reflected checker and its soundness theorem, root-section semantics, a tactic adapter, and strategy elaboration into source-level terms.

Two small core-Lean semantic combinators are included to show the final quantifier-composition shape. They are explicitly uncompiled specimens; they are not a surrogate for the missing algebraic or geometric verification. No Lean executable was available in the execution environment, and no Lean tactic comparison is reported.

2 Projection by checkable identities

2.1 What can change as the parameter moves?

Fix a polynomial $f(X, Y)$ and imagine moving x continuously along the real line. The real roots in Y can change their qualitative arrangement when a leading coefficient vanishes, when a root becomes multiple, or when roots belonging to different input factors meet. Purely X -dependent factors can independently change an atom’s sign. The certificate records a finite set of parameter polynomials whose nonvanishing excludes these events.

The producer is allowed to find these polynomials with sophisticated algebra. The checker does not trust that a returned “resultant” or “discriminant” was computed correctly. It verifies ordinary identities in $\mathbb{Q}[X, Y]$. This is the principal simplification of the trust boundary.

2.2 Source-preserving factorization receipts

The producer proposes nonconstant, nonzero factors $f_1, \dots, f_s$ and identities

$$p_k = c_k \prod_{i=1}^s f_i^{e_{ki}}, \quad c_k \in \mathbb{Q}, \quad e_{ki} \in \mathbb{N}. \quad (2)$$

The zero polynomial is represented by scalar zero with no factor powers. Nonzero constants need no factors. The checker expands every identity in its own sparse rational arithmetic and compares

it with the independently supplied source polynomial. It rejects duplicate factors, unused factors, invalid indices, zero coefficients in canonical sparse rows, and noncanonical rational encodings.

Irreducibility is *not* a checker premise. The producer uses rational factorization to obtain a convenient basis, but the necessary square-freeness and pairwise separation are established by the subsequent Bézout receipts. This removes one large algebraic algorithm from the reflected-checker obligation.

Let S denote the factors of positive Y -degree. A factor depending only on X contributes directly to the projection guard set. For $i \in S$, put $\ell_i(X) = \text{lcy}(f_i)$ and require a derivative receipt

$$A_i(X, Y)f_i(X, Y) + B_i(X, Y)\partial_Y f_i(X, Y) = d_i(X), \quad d_i \neq 0. \quad (3)$$

For every unordered pair $i < j$ in S , require

$$A_{ij}(X, Y)f_i(X, Y) + B_{ij}(X, Y)f_j(X, Y) = d_{ij}(X), \quad d_{ij} \neq 0. \quad (4)$$

All coefficients and all identities are exact. An identity is stronger evidence than a solver's declaration that a gcd equals one, while still being cheap to check by multiplication and addition.

Lemma 2.1 (The guards have the required meaning). *If $\ell_i(x)d_i(x) \neq 0$, then $f_i(x, Y)$ has its declared degree and no multiple complex root. If $d_{ij}(x) \neq 0$, then $f_i(x, Y)$ and $f_j(x, Y)$ have no common complex root.*

Proof. The leading coefficient proves the degree assertion. A multiple root would annihilate both $f_i(x, Y)$ and its derivative, contradicting (3) evaluated at that root. A common root contradicts (4) in the same way. These arguments work in $\mathbb{C}$ and hence also for real roots. $\square$

A checker must itself enumerate all required derivative indices and unordered pairs. Merely checking each supplied identity permits the producer to omit the one pair whose roots cross. In the prototype, exact coverage and order are enforced before any identity is used.

2.3 How the producer finds the receipts

Run the extended Euclidean algorithm in the univariate ring $\mathbb{Q}(X)[Y]$. For a coprime pair f, g , it returns a, b, h with $af + bg = h$, where h is a nonzero element of $\mathbb{Q}(X)$. Divide by h , collect the denominators of the resulting coefficient rational functions, and clear them with a polynomial common multiple $d(X)$. The result is $Af + Bg = d(X)$ with $A, B \in \mathbb{Q}[X, Y]$.

The prototype uses SymPy's `gcdex` over `QQ.frac_field(x)` and checks the emitted result by an unrelated sparse-identity computation. It uses `factor_list` and square-free/lcm operations only in the producer [13]. Nothing requires d to be the minimal resultant. Multiplying a valid guard and both multipliers by another nonzero parameter polynomial is sound but introduces unnecessary exceptional cells. Minimizing that redundancy is a search concern, not an acceptance condition.

```
produce_guards(factors):
    guards := all factors with y_degree = 0
    for each factor f with y_degree > 0:
        guards += leading_y_coefficient(f)
        (A, B, d) := bezout_over_Qx_then_clear(f, derivative_y(f))
        emit DerivativeGuard(f, A, B, d)
        guards += d
    for every unordered pair (f, g) of positive-y-degree factors:
        (A, B, d) := bezout_over_Qx_then_clear(f, g)
        emit PairGuard(f, g, A, B, d)
        guards += d
    return monic_squarefree_lcm(guards)
```

Over characteristic zero, distinct irreducible Y -positive factors in $\mathbb{Q}[X, Y]$ become coprime in $\mathbb{Q}(X)[Y]$, and each is coprime to its Y -derivative. Content depending only on X has already been separated. Thus the required nonzero receipts exist for the factor basis used by the unbounded algorithm.

2.4 The base decomposition

Let $\mathcal{G}$ contain every nonzero guard: pure- X factors, leading coefficients, derivative guards, and pair guards. Define

$$P(X) = \text{monic lcm}_{g \in \mathcal{G}} \text{sf}(g). \quad (5)$$

The empty lcm is 1. The checker computes this itself, so neither an omitted guard root nor an incorrect claim of square-freeness can alter the base cover.

Let $\alpha_1 < \dots < \alpha_r$ be all real roots of P . The base cells are

$$(-\infty, \alpha_1), \{\alpha_1\}, (\alpha_1, \alpha_2), \dots, \{\alpha_r\}, (\alpha_r, \infty).$$

For $P = 1$, or for a nonconstant P with no real root, the base consists of the whole line. Every open cell receives a rational sample. Every point cell is the actual isolated algebraic root, not a nearby rational approximation.

Each root is encoded by P together with rational bounds isolating it, or by an exact rational point. Its numerical cut is not the semantic boundary of a base cell: the boundary is the unique root denoted by that cut. Two different valid cuts can denote the same boundary and must produce the same truth result.

2.5 Why an open base cell is safe to sample

The following proof makes the geometric obligation explicit. It is not currently a theorem in the supplied Lean files.

Lemma 2.2 (Persistence and ordering of the roots). *Let $I \subseteq \mathbb{R}$ be a nonempty open interval containing no real root of P . For each $i \in S$, the real roots of $f_i(x, Y)$ form a fixed finite number of continuous functions on I . Across all such factors, these functions can be strictly ordered*

$$\rho_1(x) < \rho_2(x) < \dots < \rho_n(x), \quad x \in I,$$

without intersections.

Proof. All guards are nonzero on I . At a real root (x_0, y_0) , the derivative in Y is nonzero by Lemma 2.1. The real implicit function theorem therefore continues this root uniquely in a neighborhood of x_0 .

No additional real roots can appear arbitrarily close to x_0 without approaching one of the roots at x_0 . To see this directly, divide by the nonvanishing leading coefficient on a small closed neighborhood of x_0 . The remaining coefficients are uniformly bounded there, so a Cauchy root bound confines every root to a common compact interval. Any sequence of purported extra roots with parameters approaching x_0 has a subsequence converging to a root at x_0 . Simplicity and local uniqueness then contradict the extra-root assumption. The number of real roots is consequently locally constant, hence constant on the connected interval I .

At each parameter, order the roots. Local uniqueness makes each ordered root a continuous function. Roots from one factor cannot meet because they remain simple; roots from different factors cannot meet because their pair guard is nonzero. The total ordering therefore persists throughout I . $\square$

The local boundedness argument is important. Merely saying that roots cannot collide does not exclude a root escaping to infinity when the degree drops. The leading-coefficient guards are exactly what prevents that failure inside an open base cell.

Theorem 2.3 (Sign invariance of the open-cell stack). *Over I , take the root sections $Y = \rho_j(X)$ and the sectors below, between, and above them. Every original polynomial p_k has constant sign on each of these cells.*

Proof. Every pure- X factor is nonzero and has constant sign on I . Consider a Y -positive factor. On a sector it has no zero, and the sector is connected: using its continuous boundary functions, it is homeomorphic to a product of I with an interval. A continuous nonzero real-valued function has constant sign on a connected set.

On a root section, exactly one factor supplies that root. Its identity cannot change along I , because such a change would require a crossing. That factor vanishes throughout the section; every other Y -positive factor is nonzero there and has constant sign by connectedness. The factorization identities (2) transfer these conclusions, including the original multiplicities and scalar signs, to all p_k . $\square$

This argument also explains why sign rows belong to a *common* ordered union of fibre roots, rather than to several unrelated factor-specific root lists. A formula comparing two factors needs a stable alignment of their sign regions.

2.6 Exceptional points are fresh problems

At $x = \alpha_j$, any of the guards may vanish. The degree of a factor may fall; two factors may acquire a common root; an entire factor may become zero. No generic open-cell continuation argument is applied there.

Instead, specialize every factor exactly at α_j . Form the product of those specializations with positive Y -degree, omit constants and zero polynomials from that product, and take its square-free part. Isolate its complete real-root set. Evaluate the original specialized polynomials, including any zeros or constants, on every resulting section and sector. This is an exact univariate calculation over the ordered algebraic field $\mathbb{Q}(\alpha_j)$.

Omitting a zero specialization from the *root-boundary product* does not discard its contribution to the formula. The original polynomial is still evaluated in every sign row and yields sign zero. This difference is essential for a case such as $x(y^2 - 2) = 0$: at $x = 0$ the equation holds for every y , rather than having two distinguished solutions only.

2.7 The finite result and completeness statement

Theorem 2.4 (Atlas semantics). *Suppose the source identities and guard identities are valid, the base and fibre root covers are complete, and all sample sign rows are exact. Then the Boolean formula is constant on every plane cell. On each base cell, $\exists y F(x, y)$ is the disjunction of its fibre-cell truth values, and $\forall y F(x, y)$ is their conjunction. The resulting truth is constant on that base cell.*

Proof. For open base cells use Theorem 2.3; for point cells use the univariate root decomposition. Boolean connectives preserve constancy. Every fibre cell is nonempty over every parameter in its base cell, and the cells cover the whole fibre. Existential and universal quantification therefore reduce respectively to disjunction and conjunction. $\square$

Corollary 2.5 (Completeness of the uncapped mathematical algorithm). *For every input in Section 1.3, exact rational factorization, the guard construction above, complete real-root isolation, and exact algebraic sign determination produce a finite atlas deciding the closed sentence or its free-parameter truth set.*

Proof. Rational factorization is finite and supplies factors for which the required receipts exist. The projection is a nonzero univariate polynomial with finitely many real roots. Every specialized fibre also has finitely many distinct roots unless it is zero, which the construction handles explicitly. Rational and real-algebraic arithmetic decide the finitely many required equalities and signs. Theorem 2.4 then applies. $\square$

This is a completeness statement about the specified mathematical procedure, not a formal verification of the Python program. The delivered implementation imposes explicit factor, cell, degree, refinement, and isolation-node limits; exceeding them must not be interpreted as refutation. Its tests cover a modest family, and completeness does not imply that large instances are computationally practical.

3 Exact algebraic points without a false field assumption

3.1 The representation that the checker actually needs

A complete sign atlas requires more than rational arithmetic, even when all source coefficients are rational. A projection root may be irrational, and the fibre over that root may have further algebraic roots. The prototype supports precisely this two-level situation, using a small exact representation rather than importing SymPy’s algebraic-number machinery into replay.

At the base level, a real algebraic point is represented by

$$\alpha = (P, a, b), \quad P \in \mathbb{Q}[X] \text{ square-free}, \quad a, b \in \mathbb{Q},$$

where either $a < b$ and (a, b) contains exactly one root of P , with neither endpoint a root, or $a = b$ and $P(a) = 0$. The global projection polynomial is convenient because it is already bound to the certificate. It is usually reducible.

That last fact is not a cosmetic issue. The quotient $\mathbb{Q}[X]/(P)$ need not be a field, so a routine that uses an arbitrary residue class as an invertible coefficient can silently invalidate the calculation. Factoring P and selecting the irreducible factor containing α is one possible solution. The implementation instead works with fractions whose denominators are checked at the selected root.

3.2 Localized rational-function coefficients

A coefficient in a fibre polynomial is represented as

$$[n, d]_\alpha = \frac{n(\alpha)}{d(\alpha)}, \quad n, d \in \mathbb{Q}[X], \quad d(\alpha) \neq 0. \quad (6)$$

The numerator and denominator may each be reduced modulo P , because $P(\alpha) = 0$. They may share a rational-polynomial factor that is canceled, provided the denominator condition is preserved. The prototype also normalizes the denominator’s leading coefficient. It does not claim these representations are unique.

Addition and multiplication use the ordinary fraction rules. Division by $[u, v]_\alpha$ uses $[nv, du]_\alpha$ and checks $u(\alpha) \neq 0$. Equality is decided by testing

$$(nv - ud)(\alpha) = 0.$$

Thus polynomial long division in $\mathbb{Q}(\alpha)[Y]$ operates on genuine nonzero real coefficients, not on formally nonzero residue classes that might vanish at the chosen root.

Proposition 3.1 (Localized arithmetic is semantically sound). *Provided every denominator is nonzero at α , reduction modulo P , the fraction operations above, and cancellation of a common polynomial factor preserve the denoted real number.*

Proof. Evaluation at α sends multiples of P to zero. The ordinary field identities hold after evaluation because all denominators are nonzero. If h divides a nonvanishing denominator, then $h(\alpha) \neq 0$, so cancellation is valid. Equality reduces to cross multiplication for the same reason. $\square$

For a concrete control, take

$$P(X) = (X^2 - 2)(X^2 - 3), \quad \alpha = \sqrt{2}.$$

The polynomial $X^2 - 3$ is a zero divisor modulo P , but its value at α is -1 . Consequently

$$\frac{1}{\alpha^2 - 3} = -1$$

is perfectly valid. By contrast, a denominator $X^2 - 2$ must be rejected at this root. Both behaviors are exercised by the tests. This avoids asking the producer for an irreducibility proof merely to justify each division in an exceptional fibre.

The name `AFIELD` in the code denotes this evaluation field representation. It does not assert that the quotient by the supplied defining polynomial is a field. Elements associated with different root objects are not silently mixed, even when their printed polynomials happen to match; a production implementation would add an explicit checked transport operation when two representations denote the same selected root.

3.3 Exact zero first, interval sign second

An interval method alone cannot distinguish a true zero from a tiny nonzero value by refinement in finite time. The checker first decides equality algebraically.

Lemma 3.2 (Zero test at an isolated root). *Let P be square-free, and let (a, b) isolate one real root α of P . For $q \in \mathbb{Q}[X]$, put $h = \gcd(P, q)$. Then $q(\alpha) = 0$ if and only if h has a root in (a, b) .*

Proof. The forward direction follows because the irreducible minimal polynomial of α divides both P and q , hence their gcd. Conversely, a root of h in the interval is a root of P there, so it must be α , and $h \mid q$. Equivalently, one may use the standard common-root property of a polynomial gcd over a characteristic-zero field. $\square$

Since $h \mid P$, the isolating interval's endpoints are not roots of h . Its root count is either zero or one. Exact rational points use direct evaluation instead.

Once zero is excluded, rational interval evaluation and bisection determine the sign. Evaluating a polynomial on an interval containing α gives a rigorous enclosing interval. If it lies strictly above or below zero, the sign is known; otherwise refine the isolating interval. For a localized fraction, refine until the denominator enclosure excludes zero, divide the intervals with the correct order, and continue as needed.

Proposition 3.3 (Termination of uncapped sign refinement). *For a fixed nonzero $q(\alpha)$, interval evaluation of q on nested rational isolating intervals shrinking to α eventually excludes zero. The same holds for a localized fraction with nonzero numerator and denominator.*

Proof. Polynomial evaluation is continuous and the interval Horner operations converge to point evaluation as the input width tends to zero. The limiting value is separated from zero. For a fraction, first use the nonzero denominator to obtain a zero-free denominator enclosure; division is continuous there. No numerical tolerance is used. $\square$

The code uses a refinement budget. Failure to separate within that budget raises `LimitExceeded`; it is not evidence of equality or of a false assertion. This matters especially near multiple roots or when numerator and denominator expressions have large cancellation.

3.4 Sturm chains and endpoint discipline

For a nonconstant square-free polynomial q over an ordered subfield of $\mathbb{R}$, form the signed remainder sequence

$$q_0 = q, \quad q_1 = q', \quad q_{i+1} = -\text{rem}(q_{i-1}, q_i)$$

until the next remainder is zero. Let $V(t)$ count sign changes after zero entries are removed. If neither a nor b is a root of q , then

$$\#\{z \in (a, b) : q(z) = 0\} = V(a) - V(b).$$

At infinity, signs are determined by the leading coefficient and the degree parity. A constant nonzero polynomial has no roots and a one-element chain. The zero polynomial is never sent to a Sturm-chain routine as though it had finitely many roots.

This is the standard Sturm mechanism, and formal developments of the more general Sturm–Tarski theorem provide useful models for a future Lean proof [12]. The current checker computes the chain rather than trusting a serialized root count. It has not proved the Sturm theorem internally. A Lean port must discharge that theorem or use a suitable verified implementation; wrapping the Python return value is insufficient.

The wire format takes a deliberately restrictive position on endpoints. A non-point cut is open and must have nonroot endpoints. An exactly rational root uses a point descriptor $[a, a]$. Adjacent cuts must be strictly disjoint. This sidesteps several one-sided variation conventions at the certificate boundary. A producer with touching intervals can refine them before serialization.

The restriction does not lose mathematical completeness. Every distinct real algebraic root has an isolating rational interval with nonroot endpoints; rational roots may also be represented that way if the producer does not recognize them as rational. Point descriptors are an optimization, not a separate notion of algebraic number.

3.5 Complete coverage, not a plausible list of roots

A list of individually valid cuts is not necessarily complete. The checker verifies three independent properties: every cut denotes exactly one root, the cuts are in strictly increasing disjoint order, and their number equals the total distinct real-root count obtained from the Sturm sequence at $-\infty$ and $+\infty$. Together these establish a complete root cover.

Lemma 3.4 (Root-cover sufficiency). *If a square-free polynomial has N real roots, and N disjoint cuts each isolate one of its real roots, then every real root belongs to exactly one cut.*

Proof. The cuts identify N distinct roots; there are only N roots. Disjointness supplies uniqueness. $\square$

The implicit sectors before the first root, between consecutive roots, and after the last root then cover the complement. A claimed rational sector sample is checked against the actual root values using exact sign tests; being between the current numeric cut endpoints is sufficient but not required. This allows the root objects to refine their cuts during later arithmetic without invalidating an already correct sample.

3.6 Lifting over an algebraic base point

At $x = \alpha$, specialize each bivariate polynomial coefficient by coefficient:

$$f(X, Y) = \sum_j f_j(X)Y^j \quad \mapsto \quad f(\alpha, Y) = \sum_j [f_j, 1]_\alpha Y^j.$$

Trailing zero coefficients are removed using algebraic equality, so a degree drop is detected exactly. The product of all nonconstant specialized factors is made square-free using polynomial gcd arithmetic over the localized coefficient field. Its roots are isolated by rational cuts in the Y -line.

A fibre root β is therefore represented by a polynomial $q(Y) \in \mathbb{Q}(\alpha)[Y]$ and a rational isolating cut. To test the sign of another specialized polynomial g at β , compute $\gcd(q, g)$ over $\mathbb{Q}(\alpha)$ to decide zero. For nonzero values, refine the β cut and, when necessary, the base cut for α . All coefficient enclosures remain exact rational intervals.

This is enough for two-variable lifting. It is not a general tower-of-algebraic-extensions API. The `AField` constructor in the prototype requires a rational defining polynomial at the base; adding a third quantified coordinate would require a more general representation and a new projection theorem, not simply another recursive call to the existing producer.

3.7 Producer/checker independence, stated precisely

Several computations are genuinely different across the boundary. SymPy proposes multivariate factors; the checker expands sparse rational identities. SymPy proposes Bézout multipliers; the checker expands their identities. SymPy proposes base isolating intervals; the checker validates complete coverage using its own rational Sturm implementation.

Other computations are shared. The producer's exceptional-fibre root isolation, sign evaluation, and the checker use the same exact arithmetic module. Some formula and fibre-product helpers are shared as well. Disabling site packages demonstrates that replay has no SymPy dependency; it does not demonstrate independent correctness of the shared arithmetic. The differential tests use SymPy as a second implementation on specified problems, and the mutation tests exercise rejection conditions. Neither replaces a formal soundness proof.

A particularly small implementation trap concerns Python equality and hashing. Two different fraction representations may denote the same algebraic coefficient, and a coefficient may equal an ordinary rational. A representation-based hash would violate that semantic equality contract. Algebraic elements are therefore unhashable; caches use explicit representation keys that are only memoization keys, never equality proofs. Cache misses cost time but do not change the mathematical answer.

3.8 Cost centers and useful limits

If there are s positive- Y factors, the basic projection asks for s derivative receipts and $\binom{s}{2}$ pair receipts. If the projection has r real roots, there are $2r + 1$ base cells. A fibre with n distinct roots

contributes $2n + 1$ plane cells. For fixed degree and factor count these quantities are manageable, but coefficient bit growth, rational-function cancellation, and algebraic root separation can dominate even small-dimensional examples.

General CAD has severe worst-case complexity as the number of variables grows [8]. No polynomial-time claim is made here, and the prototype's favorable timings concern very small inputs. More informative budgets than a single wall-clock timeout include coefficient bit length, expanded monomial count, total Bézout receipt size, projection degree, live algebraic elements, and total refinement work. The current code has only part of that resource discipline and is not hardened against hostile certificates.

4 From sign tables to witnesses, counterstrategies, and formulas

4.1 Quantifiers are a finite game on the atlas

For base cell C_i , let $T_{ij} \in \{\perp, \top\}$ be the truth of F on its j th fibre cell. Define

$$E_i = \bigvee_j T_{ij}, \quad U_i = \bigwedge_j T_{ij}.$$

An inner existential uses E_i ; an inner universal uses U_i . An outer existential takes a disjunction over i ; an outer universal takes a conjunction. A free outer variable returns the union of the cells with the appropriate true value.

These Boolean operations are easy. The hard facts behind them are nonemptiness, complete coverage, and truth constancy for every real point in each cell. They must not be replaced by a statement that the samples passed.

The strategy obligation changes with polarity. For a true $\forall x \exists y$ sentence, every base cell needs a witness selector. For a false $\exists x \forall y$ sentence, every base cell needs a selector producing a falsifying y . For a false $\forall x \exists y$ sentence, one selected x cell with an entirely false fibre is enough to locate a counterexample parameter. For a true $\exists x \forall y$ sentence, one selected base cell with an entirely true fibre supplies an existential parameter witness.

The prototype records these choices explicitly. For an inner existential, a choice is required exactly when that fibre is true and it must point to a true cell. For an inner universal, a choice is required exactly when that fibre is false and it must point to a false cell. Unnecessary selections are rejected as a format error rather than silently given a second interpretation.

4.2 Why the sample is not the witness function

Suppose a sample fibre at $x = 0$ admits $y = 1$ for the equation $y^2 = x^2 + 1$. The stored number 1 is not a witness for every other x . The transported object is the selected root's *ordinal position*, with respect to the union of all root boundaries in that base cell.

Let $\rho_1 < \dots < \rho_n$ be those root functions. The strategy language is

$$\begin{aligned} \text{Root}(j)(x) &= \rho_j(x), \\ \text{Below}(1)(x) &= \rho_1(x) - 1, \\ \text{Above}(n)(x) &= \rho_n(x) + 1, \\ \text{Between}(j, j+1)(x) &= \tfrac{1}{2}(\rho_j(x) + \rho_{j+1}(x)). \end{aligned}$$

If there are no roots, the constant zero is a selector for the unique sector. A point base cell uses the corresponding fixed algebraic root values. In JSON, root indices are zero-based; the equations above use the more natural one-based mathematical numbering.

Every one of these functions stays in its designated fibre cell. In particular, a midpoint stays strictly between two distinct ordered roots, which matters for strict inequalities and disequalities. A rational sector sample is used only to determine that cell's sign row. The implementation emits the root-index strategy descriptor; it does not emit a runnable exact-real Lean function implementing all these root operations.

Theorem 4.1 (Strategy extraction). *For each base cell whose existential fibre is true, selecting a true root section or sector and applying the corresponding selector gives a semialgebraic witness on that entire base cell. Combining the finitely many selectors by base-cell membership gives a total piecewise semialgebraic witness whenever $\forall x \exists y F(x, y)$ is true. Dually, false universal fibres yield piecewise counterstrategies.*

Proof. Each selector lies in its chosen cell by the strict order of the root functions. Theorem 2.4 makes F constant with the desired truth there. Each root function is semialgebraic: its graph is first-order definable over the reals by a polynomial equation and the finite ordered-root condition on a semialgebraic parameter cell; real quantifier elimination makes this graph semialgebraic. This use of the classical real-QE theorem is separate from the sign-atlas soundness proof, which does not require a quantifier-free root-graph renderer. Sums, differences, rational scaling, and finite piecewise combinations preserve semialgebraicity. The base cells partition the line, so the combined choice is total on exactly the claimed domain. $\square$

The abstract assertion of semialgebraicity does not mean the prototype has simplified the output into a quantifier-free polynomial formula. It returns root-index objects and algebraic cells. A production renderer can retain these as a small root language or convert them to a more conventional formula with additional verified transformations.

4.3 Worked example: a strict band of admissible squares

Consider

$$\forall x \in \mathbb{R} \quad x \geq 0 \implies \exists y \in \mathbb{R} \quad x < y^2 < x + 1.$$

For $x > 0$, the positive roots $\sqrt{x}$ and $\sqrt{x+1}$ bound a true sector. The midpoint selector is

$$y(x) = \frac{\sqrt{x} + \sqrt{x+1}}{2}.$$

Because both endpoints are nonnegative and distinct, squaring preserves their strict order and gives the desired inequalities. At $x = 0$, fresh specialization gives the sector $(0, 1)$, whose midpoint is $1/2$. For $x < 0$, the antecedent is false, so the Boolean formula imposes no witness constraint.

The example demonstrates three features simultaneously: the witness need not lie on an equation's root section; strictness requires an open sector; and the exceptional parameter $x = 0$ is checked explicitly rather than included by continuity of a strict inequality.

4.4 Worked example: an exact disconnected parameter region

Ask for the set of real x satisfying

$$\exists y \in \mathbb{R} \quad y^2 = 2 \wedge (y - x)^2 = 2.$$

The answer is

$$x = 0 \quad \vee \quad x^2 = 8. \tag{7}$$

Indeed, y and $y - x$ independently take values in $\{\sqrt{2}, -\sqrt{2}\}$, so their difference x is 0 or $\pm 2\sqrt{2}$. Conversely, these three values admit witnesses: $y = \sqrt{2}$ at $x = 0$ or $x = 2\sqrt{2}$, and $y = -\sqrt{2}$ at $x = -2\sqrt{2}$.

The emitted projection polynomial is exactly $P(X) = X^3 - 8X$. Its three real roots give seven base cells, with truth vector

$$(\perp, \top, \perp, \top, \perp, \top, \perp).$$

All true cells are points, two of them irrational. This is not a problem that can be settled by treating open parameter intervals as the only interesting regions.

The prototype's algebraic cuts for the outer irrational roots need not display radicals. The checker validates the roots of $X^3 - 8X$ and their fibre equations directly. The concise expression (7) is an independently checked description used in the differential suite; the atlas producer itself returns the seven-cell truth structure rather than claiming to discover that simplification syntactically.

4.5 Worked example: a degree drop that is not a failure

For

$$\exists y \in \mathbb{R} \quad xy^2 + y - 1 = 0,$$

the exact feasible parameter region is $x \geq -1/4$. If $x \neq 0$, the discriminant is $1 + 4x$. If $x = 0$, the equation is linear and has the solution $y = 1$.

The certificate contains the particularly simple identity

$$(-4x)(xy^2 + y - 1) + (2xy + 1)(2xy + 1) = 4x + 1. \quad (8)$$

The leading coefficient contributes the guard x , so the projection is the monic polynomial $X(X + 1/4)$. Its point cells are $x = -1/4$ and $x = 0$. The first has the double root $y = 2$; the second has the linear root $y = 1$. The truth vector is

$$(\perp, \top, \top, \top, \top).$$

Dropping the x guard because the discriminant is already present would be a mistake in the generic continuation proof. Conversely, declaring the $x = 0$ branch unsupported would unnecessarily lose a correct positive answer. Fresh specialization handles both exceptional mechanisms with the same root-checking code.

4.6 Worked example: cubic and quintic inverse relations

The cubic relation

$$y^3 - y = x$$

is surjective, but its number of real inverse branches changes. The emitted projection is $X^2 - 4/27$. The two irrational parameter values $\pm 2/(3\sqrt{3})$ become point cells; the three open intervals between and around them have different root configurations. All five base cells have a nonempty fibre, as the stored certificate verifies.

This example is a useful stress test because the exceptional fibres are not rational-coefficient univariate problems after specialization. They require exact arithmetic at a selected irrational base point. A representation that only knows how to handle rational projection roots cannot claim to cover it.

The quintic relation

$$y^5 + y = x$$

has exactly one real root for every x , since its derivative is $5y^4 + 1 > 0$ and its values tend to opposite infinities. The general producer does not use that specialized argument. It emits a derivative guard proportional to $3125X^4 + 256$, which has no real root, and a single base cell suffices. This is a useful example of a nonconstant projection polynomial that does not require any real split.

For practical Forge use, a recognized odd-degree surjectivity theorem or monotonicity argument may be cheaper than the atlas. The point of these tests is not to assert that CAD is the best proof of every example, but to exercise a uniform certificate language that can express root witnesses without a radical template.

4.7 Worked example: discontinuity is allowed when the specification allows it

Consider

$$\begin{aligned}\forall x \exists y \quad & y^2 = x^2 + 1, \\ & (x \geq 0 \implies y \geq 1), \\ & (x < 0 \implies y \leq -1).\end{aligned}$$

A witness is $-\sqrt{x^2 + 1}$ on $x < 0$ and $+\sqrt{x^2 + 1}$ on $x \geq 0$. The jump at zero is forced by the two sign conditions: left-hand values are at most -1 , whereas the value at zero must be at least 1 .

A synthesis method that searches only globally continuous templates is solving a stronger problem than the user asked. The atlas partitions at the pure- X guard $x = 0$, chooses the appropriate branch in each cell, and handles the point separately. It does not promise continuity, differentiability, or any other regularity not present in the input.

A later regularity-aware synthesis mode would need additional boundary-compatibility constraints on the selected branches. Selecting any locally valid cell is complete for plain existence but not for existence of a continuous global selector.

4.8 Worked example: quantifier order and dual evidence

The sentence $\forall x \exists y \ y > x$ is true. The upper sector above the root $y = x$ gives the selector $y = x + 1$. The reversed sentence $\exists x \forall y \ y > x$ is false. In every base cell, the root section $y = x$ or the lower sector yields a counterstrategy.

The difference is not resolved by searching for a particularly good constant. In the first sentence, the witness may depend on the universally chosen x ; in the second, one x must survive every y . The atlas retains the quantifier prefix and extracts evidence with the correct dependency order.

Similarly, a refutation of $\forall x \exists y \ xy = 1$ selects $x = 0$ and certifies that its entire fibre is false. A refutation of $\exists x \forall y \ xy = 1$ would require a falsifying y for every possible x . Those are different certificate shapes even though both closed sentences are false.

4.9 Free-variable output and reuse by Forge

For a free parameter, the result is an exact one-dimensional semialgebraic set represented by a union of root cells and sectors. The same output can serve three different tasks: a proved guard sufficient for a desired witness, a necessary-and-sufficient parameter condition, or a counterexample region for a candidate assertion.

The worker should expose these distinct consequences through the existing obligation graph. A proved guard may help a surrounding case split; an exact equivalence may simplify a quantified assertion; a witness must be elaborated at its exact requested type. None of these operations

should implicitly turn a fragment-specific negative result into a statement about arbitrary Lean terms inhabiting an unrelated type.

5 A concrete integration plan for Lean, Forge, Leant, and Djex

5.1 Separate the executable data model from the semantic model

The polynomial wire format should not become the mathematical definition of a real polynomial. Use a small executable representation—canonical arrays of rational coefficients or sparse monomials—with a proved interpretation into Mathlib’s polynomial types. For this worker, a bivariate polynomial can be interpreted either as `MvPolynomial (Fin 2) Rat` followed by evaluation into $\mathbb{R}$, or as `Polynomial (Polynomial Rat)` with Y outermost. The latter matches the projection algorithm; the former may fit an existing reifier more naturally. Conversion between them must be justified once, not reconstructed by ad hoc casts in each certificate.

A proposed module division is:

Module	Responsibility and proof boundary
<code>Forge.Algebra.Poly2</code>	Executable sparse arithmetic; evaluation-preservation theorems
<code>Forge.Algebra.Sturm</code>	Signed remainder chains, endpoint signs, exact root counts
<code>Forge.Algebra.RootPoint</code>	Isolating cuts; root existence, uniqueness, comparisons
<code>Forge.Algebra.Localized</code>	Rational functions at a selected root; nonzero-denominator proofs
<code>Forge.Atlas.Guards</code>	Source identities, pair coverage, noncollision consequences
<code>Forge.Atlas.Geometry</code>	Root persistence, ordered sections, sign-invariant stacks
<code>Forge.Atlas.Check</code>	Reflected certificate validation and semantic soundness
<code>Forge.Atlas.Strategy</code>	Root-index selector semantics and quantifier propagation
<code>Forge.Tactic.Atlas</code>	Source reification, worker call, checked proof reconstruction

These are proposed paths, not modules already added to the user’s repository. The delivered Python modules mirror the lower computational portion of this split.

5.2 The Lean theorem that must actually be earned

Forge already has a `CertificateSpec` with a Boolean checker and a `sound` field [4]. The atlas should instantiate that interface rather than invent a competing trust architecture. For a Boolean verdict, the semantic contract can be written mathematically as

$$\text{check}(p, c) = \text{accepted}(b) \implies (\text{denotes}(p) \leftrightarrow b = \text{true}).$$

For a free-parameter result S_c , the stronger contract is

$$\text{check}(p, c) = \text{accepted}(S_c) \implies \forall x \in \mathbb{R}, \quad (Q_y y \llbracket F \rrbracket(x, y) \leftrightarrow x \in S_c).$$

A separate theorem interprets a selected root-index program as a witness or counterstrategy on the appropriate cells. Merely proving the last finite Boolean fold is correct does not establish either contract; the hard dependencies are the geometric theorem and exact algebraic arithmetic.

The included `lean/AtlasQuantifiers.lean` illustrates only the final composition step: a supplied cover plus a supplied correct witness on every cell yields $\forall x \exists y$, and supplied cellwise counterexamples refute $\exists x \forall y$. Its premises explicitly contain the missing mathematical work. There are no `sorry` placeholders, but the file was not compiled here. Its presence is not evidence that any real atlas certificate has passed the Lean kernel.

5.3 Reification must preserve the actual quantified variables

The first Lean adapter should support the exact fragment of Section 1.3, not a vaguely “mostly polynomial” approximation. It should open the requested real binders with fresh local identities, recursively recognize addition, multiplication, negation, natural powers, and exact rational constants, and return an equality theorem connecting every reified polynomial to its source expression under those binders. Boolean connectives and quantifiers require analogous semantic bridges.

An opaque expression depending on x or y cannot be turned into a new independent coefficient. For example, replacing $\sin x$ by an unrelated constant changes a quantified sentence. A source term independent of both binders but not known to be rational also lies outside this implementation’s coefficient domain. Such a term should cause a precise fragment refusal, or be handled by another explicitly proved reduction.

Division deserves a separate preprocessing rule. For the initial adapter, reject variable denominators. A later extension can split on the denominator’s sign and zero branch before clearing it, proving equivalence with Lean’s totalized division semantics. Clearing a denominator without recording its nonzero condition changes the formula at exactly the exceptional points this worker was designed to preserve.

Likewise, $\forall n : \mathbb{Z}$ is not $\forall x : \mathbb{R}$. Proving a real universal strengthening can sometimes establish an integer universal goal after a checked embedding, but real existential witnesses do not automatically become integers, and a real counterexample may not refute an integer statement. Approximation polarity belongs to the source bridge. An exact negative result is usable only for the exact encoded claim or through a separately proved implication with the correct direction.

For a local theorem context $\Gamma \vdash G$, the adapter can encode the relevant real-algebraic hypotheses as an antecedent while preserving their free-variable dependencies. It must not export a refutation of an unconstrained standalone formula as a refutation of G under stronger assumptions. The initial implementation should prefer an explicit two-binder subgoal over aggressive generalization of a large dependent context.

5.4 Do not turn real existence into an unpromised executable program

Lean’s mathematical $\mathbb{R}$ supports classical existence proofs. A root section may be defined noncomputably from its unique-root or ordered-root specification, with the atlas supplying its existence and ordering theorem. This gives a perfectly valid mathematical witness in a theorem, but it is not an executable numerical algorithm on arbitrary real inputs.

There are therefore two separate integration targets. The first is *proof automation*: prove an existential theorem and, where useful, expose a noncomputable root-index definition with its specification. The second is *computable synthesis*: accept a represented computable or algebraic input, return a root approximation or exact algebraic object, and prove its error or specification contract. The latter requires a representation-sensitive API and must not be advertised merely because a JSON file contains a selector.

For simple cases, a postprocessor may recognize a root as `Real.sqrt` or another existing library construction and replace the general selector with a smaller term. Acceptance must prove that

this simpler term has the same required property. The simplifier need not establish syntactic identity with the original root-index program; a direct specification proof can be cheaper.

5.5 Worker scheduling and useful partial outputs

Keep the initial tactic integration coarse-grained. The controller extracts a candidate bivariate subproblem, calls the external producer, and asks Lean to check the returned object. Only after checking does it publish a result to the ordinary Forge fact/obligation mechanism. There is no need to modify `grind`'s internal saturation state for the first milestone.

A concrete trigger is a remaining existential real witness constrained by a nonlinear equality, or a universally quantified nonlinear assertion for which a finite exact parameter condition would be useful. A cheap filter records the number of variables, maximal degrees, monomial counts, and whether an exact source bridge was constructed. Reject the fragment early when it contains a third essential variable; do not let an external solver guess what was meant.

Useful outputs are not limited to closing the entire goal. An exact free-parameter result can become a local case split. A negative region can guide typed candidate synthesis toward a missing branch. A root section can be passed to an existing arithmetic engine together with its defining equation and sign facts. A cell boundary can suggest an algebraic equality that another worker proves more cheaply. All such exports remain ordinary proved source-level facts.

5.6 A specific Leant/Djex cooperation loop

Leant's named behavioral queries associate an exact candidate with an exact assertion and distinguish a proved assertion, a proved negation, and an inconclusive proof attempt [5]. The atlas can extend that final proof stage for assertions which reify to the supported fragment. For example, a proposed function represented by a polynomial relation and a branch condition can be checked for all real inputs, not merely on a finite example set.

A candidate loop should retain the existing typed owner and provider selection. For each candidate, first elaborate its full type; next obtain a source theorem describing its relevant behavior as $F(x, y)$; then request an atlas for the exact assertion or its negation. A checked counterexample parameter may reject that candidate. An atlas timeout may only leave it inconclusive. The atlas must not authorize filtering an entire type-directed search branch merely because one tested implementation failed.

A stronger future loop can synthesize branches. Starting from a candidate that fails on a parameter cell, add that exact cell as a behavioral obligation, ask Djex for an alternative term under its local hypotheses, and assemble the two candidates with a checked case split. The finite atlas supplies the geometric partition; Djex supplies type-correct terms in each branch; Lean verifies the assembled function. This is a concrete way to combine algebraic witness discovery with higher-order typed composition without erasing universe, dictionary, or nominal-provider distinctions already preserved by the neighboring projects.

The delivered prototype stops before this host-level loop. Its strategies are algebraic selectors, not arbitrary higher-order Lean terms, and the unresolved Church-encoding and universe-composition work in Leant/Djex is not claimed to be solved by this proposal.

5.7 Libraries and theorem dependencies worth reusing

SymPy: the current producer. Version 1.14.0 was used. The concrete operations are rational multivariate factorization, extended gcd in $\mathbb{Q}(X)[Y]$, denominator lcm, square-free projection,

and rational real-root isolation. The checker is independent of these imports. The independent test oracle uses exact univariate inequality reduction and root counting [13].

FLINT: a performance-oriented producer backend. Its rational multivariate polynomial and exact algebraic-number facilities are natural candidates for replacing the current producer’s arithmetic bottlenecks. The documented `fmpq_poly` representation and `qqbar` algebraic-number representation cover relevant building blocks [14, 15]. These libraries would remain untrusted proposers, emitting the same sparse identities and isolating cuts. This article does not claim that every required extended-gcd-over-function-field operation is a single FLINT call, nor that the current Python bindings expose every C capability. No FLINT backend was executed.

LibPoly: a nonlinear-reasoning backend candidate. LibPoly targets polynomial manipulation for symbolic reasoning engines and offers algebraic/root-oriented operations [16]. It is worth evaluating for more incremental lifting and SMT-style integration. Again, the acceptance language should remain the certificate format rather than a library-specific truth flag. No ABI, license integration, or baseline compatibility test was performed here.

Mathlib: semantic infrastructure, not an assumed ready atlas checker. The existing polynomial derivative, root, topology, and implicit-function modules are concrete starting points. In particular, `Polynomial.continuous_aeval` and the `ImplicitFunctionData` interface address ingredients of the persistence argument; the polynomial limit library supplies useful leading-term asymptotics [17, 18, 19]. These are mathematical building blocks. They do not by themselves turn an executable signed-remainder implementation into a checked root counter.

An upstream Sturm candidate, with a status caveat. Mathlib PR 42558 proposes `Polynomial.IsSturmSequence.count_roots_between` on half-open intervals. At the retrieved status it was closed on 10 August 2026 without merging; its head is `df3f4a5620232d6164fb8cbb0177876f5a29de18`. The inspected module defines an abstract sign-sequence interface and describes a separate certificate layer [20, 21]. This is a concrete reuse candidate to review and compile against Forge’s pin, not an available theorem that this delivery has imported. Our nonroot-endpoint convention would need a proved conversion from its half-open formulation.

Isabelle/AFP: formalization references. The Sturm–Tarski development and the complete real-quantifier-elimination work by Kosaian, Tan, and Platzer demonstrate formal routes through the relevant mathematics [12, 11]. They are especially useful for decomposing sign determination and quantifier semantics into reusable lemmas. They are not drop-in Lean libraries, and the present implementation does not call Isabelle or import its proof objects.

5.8 Two concrete next algorithms, beyond the baseline prototype

Status. The optimizations in this subsection are designs, not measured features of the delivered worker.

Algorithm A: polarity-directed certificate slicing. The version-1 checker deliberately validates every cell. Once its theorem is formalized, a version-2 certificate can retain only what the requested polarity needs. For true $\forall x \exists y$, retain a complete base cover and one certified true section or sector over each base cell. For false $\forall x \exists y$, retain a single exact x and a complete false fibre cover. For true $\exists x \forall y$, retain one exact x and a complete true fibre cover. For false $\exists x \forall y$, retain a complete base cover and one false selector per base cell.

Start by slicing a fully generated atlas and proving a separate verifier sound; this measures proof-object and replay savings without changing search. Next generate the required branches lazily. The certificate must still justify each retained root’s existence, the chosen sector’s sign,

and the required covers. It is unsound to obtain a smaller object by merely deleting rows from the version-1 format; the mutation suite explicitly rejects that.

Algorithm B: Boolean-implicant-guided local projection. At a sample where F is true, derive a small conjunction of signed atoms sufficient for F . A propositional checker verifies that implication. Build the local algebraic cell using only the factors necessary for those atoms, with explicit guards and root-order facts. For a false sample, use a conjunction sufficient for $\neg F$. Search the uncovered parameter region for the next sample and repeat, maintaining a checked one-dimensional union of covered cells. Any newly discovered event splits the affected cells; no sampled agreement licenses extension past an unproved boundary.

The first useful implementation can restrict local cells to rational endpoints and already isolated projection roots. It can fall back to the full factor set whenever a local guard cannot certify the desired interval. That yields a precise, testable progression toward cylindrical algebraic coverings rather than an instruction to “use smarter CAD.” Proof-oriented local-cell and quantified-covering literature gives relevant designs to compare [9, 10]. The full atlas remains the completeness fallback and the reference oracle for small tests.

5.9 Implementation milestones and anti-substitution acceptance tests

Milestone 1: arithmetic replay in Lean. Port sparse identities and rational Sturm checking, with a theorem that every accepted cut denotes exactly one root and that the checked list is complete. Acceptance must include rational endpoint cases and a polynomial with an omitted real root rejected. Passing only arithmetic identities does not close this milestone.

Milestone 2: algebraic exceptional fibres. Port the localized coefficient semantics and its exact zero/sign operations. Accept the reducible-polynomial denominator control and the cubic discriminant fibres. A rational-parameter-only checker does not count as acceptance.

Milestone 3: geometry and quantifiers. Formalize root persistence and the sign-atlas theorem, then reconstruct the free-parameter result of the two-shifted-squares example. The proof must include the isolated irrational point cells. A numerical oracle or a proof for rational x is not a substitute.

Milestone 4: source bridge and tactic entry. Reify actual Lean goals with exact binder identities; reject third-variable and nonpolynomial dependencies; run the external worker; replay and commit a kernel-checked theorem through Forge’s existing policy. The primary acceptance case is a whole quantified source theorem, not an unconnected polynomial lemma.

Milestone 5: strategy terms and composition. Elaborate a branch-dependent witness and its specification, exercise the quantifier-order negative control, and integrate a checked free-parameter guard with another Forge worker. Only after this should measurements be described as Lean solved-goal results.

These milestones isolate the difficult work instead of hiding it in a generic `sound` assumption. They also permit useful partial delivery: a verified rational root-cover component benefits existing Forge workers before the entire atlas tactic is ready.

6 Executed experiments and what they establish

6.1 Environment, corpus, and evidence units

The implementation was executed with Python 3.13.5, SymPy 1.14.0, and pytest 9.0.2 on an x86-64 Linux environment. The seeded randomized checks use seed 20260915. Exact versions,

platform strings, per-case timings, and raw records are included under **results/**. No Lean or Lake executable was found, so no Lean elaboration, kernel checking, or tactic benchmark was run.

The primary corpus contains 24 hand-designed examples: 19 closed sentences and five free-parameter problems. They produce 356 plane cells in total. The largest projection degree in this corpus is four; the largest displayed inverse-relation degree in Y is five. These are deliberately small examples selected to expose semantic edge cases, not a representative distribution of Lean arithmetic goals.

Experiment	Count	Observed result
Primary atlases	24	All replayed; expected closed verdicts matched
Affine-transformed closed cases	76	All expected verdicts matched
Corrupted certificates/problems	540	All rejected
Unit and regression tests	42	All passed
Rational polynomial root problems	250	Exact SymPy root-count agreement
Localized algebraic expressions	200	194 sign agreements; 6 zero denominators rejected
Independent rational fibre queries	336	All agreed; no oracle-unknown results
Exact free-parameter cells	25	All matched five independent region descriptions
Transported selector-cell probes	237	Required witness/counterexample truth held

These rows must not be added into a purported number of independently solved theorems. The 42 tests include replay of the 24 primary examples. The 76 transformed cases preserve those examples' closed truth by construction. The rational fibre and selector probes overlap in parameter values. Mutation controls test rejection rather than theorem discovery. Each row has a distinct purpose and an explicitly recorded unit.

6.2 Primary acceptance and independent parameter-region checks

The primary producer emits a source problem, a complete certificate, and a strategy descriptor for every example. The replay checker validates all finite algebraic premises and the quantifier folds. Expected closed verdicts were supplied independently in the fixture definitions; the free-parameter cases additionally use the following independent exact answers:

Quantified relation	Exact parameter region
$\exists y : y^2 = 2 \wedge (y - x)^2 = 2$	$x = 0 \vee x^2 = 8$
$\exists y : y^2 \leq x \wedge (y - 2)^2 \leq x$	$x \geq 1$
$\forall y : y^2 + xy + 1 > 0$	$x^2 < 4$
$\exists y : xy^2 + y - 1 = 0$	$4x + 1 \geq 0$
$\exists y : y^2 = x \wedge y \neq 0$	$x > 0$

The region checker evaluates these descriptions at the exact algebraic base roots and at the rational sector samples. It checks all 25 base cells, including strict endpoints. For example, the quadratic-positive region excludes $x = \pm 2$, while the quadratic-existence region includes its

double-root boundary. The quadratic-positive example is also a standard real-QE illustration, not a new mathematical result [8].

The overlapping-interval answer has an elementary independent proof: if both y^2 and $(y - 2)^2$ are at most x , then at least one of $|y|$ and $|y - 2|$ is at least one, so $x \geq 1$; conversely $y = 1$ works. This gives a useful oracle independent of the projection construction.

6.3 Differential arithmetic and fibre oracles

For 250 random rational polynomials of degree one through six, the checker arithmetic isolates the distinct real roots. SymPy independently counts them and validates every emitted cut. The test compares exact counts and rational intervals rather than floating approximations.

The 200 localized-coefficient trials use the reducible defining polynomial $(X^2 - 2)(X^2 - 3)$ at the selected root $\sqrt{2}$. Random numerator and denominator polynomials produce 194 defined expressions whose signs agree with exact SymPy simplification. Six denominators vanish at the selected root and are rejected. The dedicated zero-divisor control also checks that a denominator vanishing at a *different* root is allowed.

For each of the 24 primary problems, the independent fibre oracle fixes x at fourteen rational probes ranging from -5 to 5 , including 0 , $\pm 1/4$, and $\pm 1/2$. It uses SymPy’s exact univariate inequality reduction on a disjunctive normal form, with universal quantification handled by negating an existential counterexample query. All 336 attempts agree with the corresponding atlas truth. The runner has an explicit oracle-unknown bucket for unsupported reductions; no such result occurred in this run. This check validates whole one-dimensional fibres at off-sample parameters, not merely a finite list of y values.

Separately, the selector transport test reconstructs the fibre at the same parameter probes and checks the originally selected root ordinal or sector. It has 237 applicable probes and all have the required truth. This test uses the replay arithmetic rather than an independent arithmetic backend. Its role is to catch indexing or transport mistakes that an inner-quantifier truth check alone could miss. It does not evaluate a generated Lean witness program, because no such program has been generated.

6.4 Metamorphic cases and replay without search libraries

Four invertible affine substitutions are applied to each of the 19 closed examples:

$$(x, y) \mapsto (x + 1, y - 1), \quad (-x, y + 2), \quad (2x - 1, -y + 1), \quad (-2x + 3, 2y - 3).$$

Each coordinate transformation is a bijection of $\mathbb{R}$, so it preserves the truth of either quantifier on that coordinate. All 76 resulting problems are solved and replayed, with their expected closed truth inherited through this elementary equivalence. Negative scales are included to exercise order changes without hand-modifying the formulas’ inequalities.

Both the primary and transformed certificate corpora are replayed with `python -S`. The recorded replay reports confirm that site packages are disabled and no `sympy` module is loaded. These runs validate the package boundary: a saved certificate can be audited without the producer or its computer-algebra dependency. They do not validate the checker by a second implementation.

6.5 Corruption controls

The 540 targeted mutations include missing base stacks, omitted base or fibre roots, invalid cuts, altered sign rows, wrong projected truth, missing derivative or pair guards, zero guards, modified

factorization scalars, absent factor powers, out-of-range selections, malformed quantifiers, and invalid atom references. Canonical representation controls reject floats masquerading as exact rationals, strings such as 0/1, and Boolean values masquerading as integer sign or index entries.

Each mutation starts from an accepted original problem/certificate pair and changes a condition that should invalidate acceptance. All are rejected. This is evidence that the intended rejection checks are exercised, not an estimate of resistance to arbitrary malicious inputs. No full adversarial decoder audit, fuzzing campaign over arbitrary byte strings, or denial-of-service hardening is claimed.

6.6 Observed timings, with the right denominator

In the recorded primary run, the sum of the 24 producer times is approximately 0.507 seconds and the sum of their replay times is approximately 0.239 seconds. The 76-case transformed run takes approximately 2.63 seconds in total. These are single-environment observations for tiny inputs; they exclude any Lean proof reconstruction and are not comparisons against `grind`, `nlinarith`, an industrial CAD solver, or Forge’s other prototypes.

The certificate structure is more informative than these timings. The tests show that a small exact implementation can represent algebraic witnesses, isolated infeasible parameters, strict sectors, and degree drops in one auditable language. A meaningful performance evaluation must later measure coefficient growth, cell growth, replay proof size, and end-to-end Lean resource consumption on independently selected goals.

6.7 A preserved failure and a code-review repair

The first differential run failed before reaching the full fibre oracle: the symbolic conversion of an empty sparse polynomial returned Python’s integer zero, which has no SymPy `subs` method. The conversion now starts its sum with SymPy’s exact zero. The original traceback is retained in `results/differential-initial-failure.log`; the successful rerun is separate. No check was weakened to obtain the successful result.

A separate code-review repair removed a misleading algebraic-element hash and replaced memoization with explicit representation keys, as described in Section 3. The final unit tests also cover the zero-polynomial conversion, endpoint convention, reducer behavior, duplicate JSON keys, and a forced resource-limit outcome. The report does not count either repair as evidence of a previously accepted invalid mathematical certificate; the observed failure was a test-interface exception.

6.8 What remains unestablished

The Python arithmetic and checker are not formally verified. The producer and checker share some computational code. The corpus is small and correlated. There is no performance comparison with existing Lean tactics, no generic multivariate CAD extension, no generated Lean real-root strategy, and no kernel-checked atlas certificate. External SymPy factorization is not bounded by the checker’s internal arithmetic counters. Some representation caps are rejection limits rather than a fully uniform resource-accounting system.

These limitations identify the next engineering work; they do not reduce the mathematical content of the proposed fragment. The useful result of this delivery is an exact, executable baseline with a concrete certificate theorem and failure-sensitive tests. It gives Forge a larger witness language and a nonlinear quantifier mechanism that can now be developed against real artifacts rather than an architectural sketch.

A Reproduction and certificate specification

A.1 Commands

The producer requires SymPy; replay does not. Run these commands from the archive's `prototype/` directory:

```
python -m pip install -r ../requirements.txt
python run_examples.py
python -S verify_corpus.py
python differential.py
python mutation_tests.py
python metamorphic.py
python -S verify_corpus.py ../metamorphic-corpus
python strategy_probes.py
python -m pytest -q
```

The archived results are the observations reported in this article. Re-running the scripts may replace result JSON files and alter timings or valid isolating cuts. Preserve the archive's original result directory when a bit-for-bit record of the delivered run is needed. Different valid cuts are not different mathematical answers.

An individual input can be solved and independently replayed with:

```
python -m atlas solve ../examples/surjective_cubic/problem.json \
  --output /tmp/cubic-certificate.json
python -S -m atlas verify ../examples/surjective_cubic/problem.json \
  /tmp/cubic-certificate.json
```

A successful verification exit code means the certificate is accepted, including a certificate for a false encoded sentence. Inspect `closed_value` to learn the sentence's truth. `null` means the outer variable is free, with its truth set in `truth_by_x`; it is not an unknown closed verdict. An arithmetic `LimitExceeded` outcome is reported as `UNKNOWN`. Other invalid input or failed acceptance returns a nonzero exit code and no accepted verdict.

The JSON decoder rejects duplicate object keys and nonfinite JSON constants. The checker performs mathematical and canonical-form validation after decoding. These controls are useful but not a claim of complete hostile-input hardening; notably, large expressions can still be expensive before a later cap is reached.

A.2 Problem format

A problem object has schema `forge.atlas.problem/1`, a diagnostic `name`, `outer` and `inner` quantifiers, a list of polynomials, and a formula tree. Sparse polynomial terms are sorted triples $[i, j, c]$ for cX^iY^j , with nonnegative integer exponents and a nonzero canonical rational string. The empty term list is zero. For example,

```
{
  "schema": "forge.atlas.problem/1",
  "name": "inverse_hole",
  "outer": "forall",
  "inner": "exists",
  "polynomials": [[[0, 0, "-1"], [1, 1, "1"]]],
  "formula": ["eq", 0]
}
```

encodes $\forall x \exists y (xy - 1 = 0)$. Rational strings such as "-3/7" are accepted; floats, "2/2", and "0.0" are not. Atom indices are zero-based. Formula nodes are Boolean literals, sign tests, or **and**, **or**, **not**, and **implies**. The checker validates every child even if another child determines the Boolean result, so a short circuit cannot hide an invalid subtree.

A.3 Certificate fields and their obligations

Field	What replay verifies
factors	Canonical nonzero nonconstant sparse polynomials; no duplicates
factorizations	Every original polynomial equals its scalar times its factor powers; all factor uses are accounted for
derivative_guards	Exactly one receipt for each positive- Y factor; exact identity and nonzero guard
pair_guards	Exactly every unordered positive- Y pair; exact identity and nonzero guard
projection	Exact monic square-free lcm of all guards; dense coefficients in increasing degree
base_roots	Complete ordered isolating cuts for real projection roots
stacks	Exactly one stack for every base section and sector
sample	A rational sample strictly inside an open base sector, or null for a root section
y_roots	Complete root cover of the square-free specialized fibre-boundary product
y_samples	One valid rational sample per open fibre sector
signs	Exact signs of every original specialized polynomial on each section/sample
truths, value	Formula evaluation and the inner quantifier fold
selection	A correctly typed index of a required true/false fibre cell, or null when no selector is required
truth_by_x	The complete vector of inner quantified truth values
closed_value	The appropriate outer fold, or null for free x
selected_x	A valid outer witness/counterexample cell exactly when required

The arithmetic routines use dense low-degree-first tuples internally for univariate operations. Bivariate source and identity checks use sparse dictionaries. The file format does not serialize implementation-specific algebraic-field objects: they are reconstructed from the validated base root and the source factor coefficients. This avoids trusting a foreign minimal-polynomial cache or a mutable library handle.

A.4 Checker order

```

verify(problem, certificate):
    validate_problem_and_boolean_AST(problem)
    check_all_source_factorizations()
    check_derivative_coverage_and_identities()
    check_pair_coverage_and_identities()
    recompute_projection_from_all_required_guards()
    base_roots := verify_complete_root_cover(projection)

```

```

require exactly 2 * len(base_roots) + 1 stacks

for each base cell:
  x := checked_rational_sample_or_selected_algebraic_root()
  fiber_poly := squarefree_product_of_nonconstant_specialized_factors()
  y_roots := verify_complete_root_cover(fiber_poly)
  check_sector_samples_against_actual_roots()
  recompute_all_original_polynomial_signs()
  recompute_formula_truths_and_inner_fold()
  validate_witness_or_counterexample_selection()

recompute_outer_fold_or_free_parameter_truth_set()
validate_outer_selection()
return accepted_encoded_result

```

A production reflected checker may replace recomputation with smaller arithmetic receipts where profitable. For example, a gcd could be accompanied by divisibility identities and a coprimeness witness rather than recomputed by Euclidean reduction. Such a change requires a new soundness argument for the optimized check, not a trusted hint that the gcd routine probably agrees.

B Primary corpus inventory

The following table is generated from the recorded primary results. “Free” denotes a parameter-region problem rather than an unresolved Boolean answer. The archive includes each formula and certificate, not just the summary.

No.	Example	Base cells	Plane cells	Verdict
1	algebraic positive	1	7	True
2	square root conditional	3	13	True
3	square root unconditional	3	9	False
4	strict square band	5	25	True
5	inverse hole	3	7	False
6	inverse guarded	3	7	True
7	vanishing content	3	21	True
8	two shifted squares	7	55	Free
9	overlapping intervals	5	31	Free
10	uniform positive quadratic	5	17	Free
11	choose positive quadratic	5	17	True
12	surjective cubic	5	23	True
13	surjective quintic	1	3	True
14	repeated zero curve	3	9	True
15	strict repeated zero curve	1	3	False
16	negative algebraic obstruction	7	33	False
17	quantifier order forall exists	1	3	True
18	quantifier order exists forall	1	3	False
19	discontinuous choice	3	23	True
20	empty and constant atoms	1	1	True
21	constant false	1	1	False
22	degree drop at zero	5	17	Free
23	strict and disequality	3	13	Free
24	algebraic section only	5	15	True

C Review record and artifact map

C.1 Repository snapshots

Forge

58ea206bd7ad501b930add568151217bcc7f82a2. Reviewed README and status material, the complete merged nonlinear chapter, docs/IDEAS-FROM-LEANT-DJEX.md, lean/Forge/Design/Contracts.lean, and lean/Forge/Closure/Covers.lean. The read scope is narrower than an audit of every archived proposal.

Leant

6bf05ad78c467989e68290f2d08bbbed40802d485. Reviewed README and the behavioral-synthesis documentation. The latter's inspected blob has SHA 6b1f20e624d8e87bc6edbbba3651fab5a2a3e21d.

Djex

e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5. Reviewed README and used Forge's existing neighboring-project analysis to avoid duplicating already adopted synthesis/ownership ideas. The inspected README blob has SHA 12fec0edf075937f2eb42125e800845a480d4b76.

No repository writes were made. The archive is a standalone extension proposal and prototype, not a patched checkout of Forge. Online library documentation was used to identify concrete integration candidates; compatibility with Forge's pinned Lean/Mathlib baseline remains to be tested.

C.2 Files in the companion archive

article/

This article, its modular LaTeX sources, generated corpus table, and final PDF. Build with `latexmk -pdf atlas.tex`; no external font files are required or included.

prototype/atlas/

Exact arithmetic, sparse polynomial operations, strict JSON loading, producer, checker, examples, and the command-line interface.

prototype/tests/

Unit/regression tests. Standalone runners next to this directory generate examples and run mutation, differential, metamorphic, selector, and dependency-free replay checks.

examples/ and metamorphic-corpus/

Twenty-four primary and seventy-six transformed problem/certificate pairs. Primary examples additionally contain selector descriptors.

results/

Environment metadata, per-case records, independent checks, corruption results, dependency-free replay reports, test logs, and the preserved initial differential failure.

lean/

Two uncompiled semantic composition lemmas with an explicit status note. No reflected algebraic checker is hidden in this directory.

docs/

Source review record, implementation status, and run history. These distinguish executed features from suggested future optimizations.

D Closing recommendation

Add this as a bounded, optional *algebraic quantifier worker*, not as the universal executor of Forge. Its first substantial acceptance target should be a fully kernel-checked parameter-region theorem with an isolated irrational feasible point. That target forces the implementation to earn its source bridge, complete root coverage, exceptional-fibre arithmetic, and quantifier semantics together.

Once that boundary is established, the same machinery can supply algebraic witnesses, exact counterexample regions, branch hypotheses for typed synthesis, and smaller facts for the other Forge workers. The significant capability gain is not the word “CAD” in the architecture diagram. It is a checked path from a nonlinear quantified source assertion to a complete finite geometric object, and from that object back to a proof or a correctly dependent witness.

References

- [1] Vladimir Reshetnikov, *Forge: implementation status*, reviewed commit 58ea206bd7ad, 2026. Pinned source: docs/STATUS.md.
- [2] Vladimir Reshetnikov et al., *Forge merged article, nonlinear arithmetic chapter*, same commit. Pinned source: article/sections/07-nonlinear.tex.
- [3] Vladimir Reshetnikov, *Ideas from Leant and Djex*, same commit. Pinned source: neighboring-project analysis.
- [4] Vladimir Reshetnikov et al., *Forge certificate contracts*, same commit. Pinned source: Contracts.lean. See also Covers.lean for the existing discrete cover interface.
- [5] Vladimir Reshetnikov, *Leant: behavioral assertions during synthesis*, reviewed commit 6bf05ad78c46, 2026. Pinned documentation.
- [6] Vladimir Reshetnikov, *Djex README and capability description*, reviewed commit e8778f4ebd63, 2026. Pinned README.
- [7] George E. Collins, “Quantifier elimination for real closed fields by cylindrical algebraic decomposition,” *Automata Theory and Formal Languages*, LNCS 33, pp. 134–183, 1975. doi:10.1007/3-540-07407-4_17. Historical source; bibliographic details cross-checked through England’s survey.
- [8] Matthew England, “Recent Developments in Real Quantifier Elimination and Cylindrical Algebraic Decomposition,” extended abstract, CASC 2024. arXiv:2407.19781v1.
- [9] Jasper Nalbach, Erika Ábrahám, Philipp Specht, Christopher W. Brown, James H. Davenport, and Matthew England, “Levelwise construction of a single cylindrical algebraic cell,” *Journal of Symbolic Computation* 123, 102288, 2024. arXiv:2212.09309; doi:10.1016/j.jsc.2023.102288.
- [10] Jasper Nalbach and Gereon Kremer, “Extensions of the Cylindrical Algebraic Covering Method for Quantifiers,” 2024. arXiv:2411.03070.
- [11] Katherine Kosaian, Yong Kiam Tan, and André Platzer, “A First Complete Algorithm for Real Quantifier Elimination in Isabelle/HOL,” 2022 preprint, 2023 publication. arXiv:2209.10978v2. See also AFP: Quantifier Elimination Hybrid.
- [12] Wenda Li, *The Sturm–Tarski Theorem*, Archive of Formal Proofs, 2014, with subsequent revisions. AFP entry and theory sources.
- [13] SymPy development team, *Polynomials Manipulation Module Reference*, version 1.14.0. Official module reference. The execution version is pinned separately in the archive.
- [14] FLINT development team, *Multivariate polynomials over the rational numbers: fmpq_mpoly*. Official documentation. Documentation discovery only; no backend execution in this delivery.

- [15] FLINT development team, *Algebraic numbers represented by minimal polynomials and isolating intervals: qqbar*. [Official documentation](#).
- [16] SRI International, *LibPoly: a library for manipulating polynomials*. [Official repository and documentation](#). Evaluated here as a backend candidate, not integrated.
- [17] Mathlib contributors, *Mathlib.Topology.Algebra.Polynomial*. [Official generated documentation](#).
- [18] Mathlib contributors, *Mathlib.Analysis.Calculus.Implicit*. [Official generated documentation](#).
- [19] Mathlib contributors, *Mathlib.Analysis.Polynomial.Basic*. [Official generated documentation](#).
- [20] Andrew McRoberts, “feat(Analysis/Polynomial): add Sturm’s theorem for real polynomials,” Mathlib PR 42558, created 8 August 2026, closed without merging 10 August 2026. [Pull request and status](#). Closure is not an assessment here of mathematical correctness.
- [21] Andrew McRoberts, *Mathlib/Analysis/Polynomial/Sturm.lean*, proposed branch head [df3f4a562023](#), 2026. [Pinned proposed source](#). Only its interface and introductory portion were reviewed; it was not compiled or comprehensively audited for this article.